Using the Lottery Ticket Hypothesis to Decrease the Size of LLMs

Author: Len Washington III (A20484223)

Artifacts: lwashington3/cs595-2-project

lwashington3/lottery-ticket-hypothesis-for-llm

1. Motivation

Transformers and Large Language Models have allowed for more efficient execution of NLP tasks. One such example where LLMs have been implemented is the EXSCLAIM pipeline,[1] which utilizes LLMs to split captions from research papers to match with any subfigures within the overall (master) figure. However, the biggest problem with running EXSCLAIM on its current servers is that there are no GPUs, which significantly reduces how fast the pipeline can run. Coupled with the dozens of captions that may need to be separated, the Caption Distributor is easily the largest bottleneck within the pipeline. The default LLM (Llama 3.2) can do a fair job of splitting these captions, though it will either need to be replaced by a better model (which would have to be larger) or have some type of improvement done. To address these issues, a method was sought out to increase the inference speed without disrupting the current accuracy of the models. A pruning technique known as the Lottery Ticket Hypothesis was chosen, since it had claims of reducing a neural network's weights by 95% while still maintaining similar accuracies as the unpruned version.

2. Background

The Lottery Ticket Hypothesis states that "a randomly-initialized, dense neural network contains a subnetwork that is initialized such that–when trained in isolation–it can match the test accuracy of the original network after training for at most the same number of iterations."[2] Essentially, by training a randomly-initialized neural network, a subnetwork, or winning ticket, can be discovered and retrained from the beginning, matching the accuracy of the original model.

The paper describes two methods towards pruning a network, one-shot pruning, and iterative pruning. For one-shot pruning, the trained network is pruned once, with $p\%$ of weights being masked out (pruned) while the reset are reset to the initially-random values. There is also iterative pruning, which repeatedly trains, prunes, and resets the network over n rounds, each time pruning another $p^{\frac{1}{n}}\%$. Brix, Bahar, and Ney describe further methods for utilizing the Lottery Ticket Hypothesis with transformers. One such method recommends applying the computed mask m to one of the earlier checkpoints from when initially-randomized model is trained, rather than the initial model. However, the paper does not go into more detailed implementation of such methods.

Since EXSCLAIM's pipeline was summarizing scientific captions into a list of strings, a similar dataset was sought with hopes of further use for EXSCLAIM. As such, one dataset

stood out, MMInstruction/ArxivCap,[4] which was designed training parser to understand Arxiv. The 755 GB dataset was filtered through, retaining only what was necessary, before re-uploading it to HuggingFace. This dataset, lwashington3/ArxivCap-Minimal, however, did not seem to work well with most LLMs, and was replaced for the duration of the project with abisee/cnn_dailymail.

3. Methodology

To utilize the Lottery Ticket Hypothesis with an LLM that I did not initially train, nor possess the training dataset, several steps are required. The first is to obtain the raw weights of a given LLM, as well as a dataset that would be used to fine-tune the model. The model will also be preprocessed utilizing this dataset, to help bridge any assumptions made by (2) having the initial training dataset for the pruning process. Once the model has been preprocessed, each layer within the model will be subjected to magnitude pruning, where the weights are sorted by magnitude, then the smallest $p\%$ of parameters are masked out (to simulate being removed).

To evaluate the results, a pipeline was constructed to run through the original (unpruned) model, as well as several pruned models using the held-out portion of the dataset. Each pruned model was recomputed by applying the mask to its base model.[1] Once the model has been masked, the model is called once using HuggingFace's native implementation with timers set to measure how long it takes for the first token to be generated, as well as the overall run time. After that, the model is called again, this time using PyTorch's implementation, to receive the logits from the output which can be used to calculate the Perplexity of the prediction. The last metric, the ROGUE-L F1 score, can be calculated with either the true value from the dataset or the predicted value of the baseline as the target. For the following results, the ROGUE-L was calculated using the predicted value of the baseline as the target value, since the purpose of this is to see how close the results of a pruned model can get to the original model.

[1]The training pipeline could have saved the masked model directly, but it was more efficient to have a single base model saved, and apply several mask files (with a smaller datatype and storage requirement) than to save multiple larger models.

4. Experimental Setup

Table 1: Experimental Specifications

GPU	NVIDIA A100 80GB PCIe
CPU	AMD EPYC 7763 64-Core Processor
Cores	256
RAM	503.414455 GB
Disk Space	418 GB
Chameleon Cloud Site	CHI@UC
Chameleon Cloud Node Name	gigaio-compute-04

Python 3.14.4 was the only language involved, and the software stack was mostly composed of the HuggingFace suite, namely Transformers, Datasets, and Safetensors (the library used to access the raw tensor parameters). The full list of libraries and versions can be seen in Appendix Section A. The baseline model that was used for pruning and evaluation was google/gemma-3-1b-it (Gemma 3) along with abisee/cnn_dailymail (CNN DailyMail) as the dataset. The metrics used to evaluate the effectiveness of pruning were Time-to-First-Token (TTFT), Time per Output Tokens (TPOT), ROGUE-L F1 score, and Perplexity.

5. Results

First, it should be noted that the original Gemma 3 model was not accurate when it came to summarizing the CNN DailyMail dataset. These low results led me to substitute in Gemma's responses for any metrics that required the true value (namely ROGUE-L) so the pruned models could be judged more accurately.

The results for the experiment turned out rather well, with the pruned models taking less time to run, while being able to produce the exact same response as the original model.[2] The results for the first row of CNN DailyMail's test set that were and were not trained can be found in Tables 2 and 3, as well as Figures 1-4. The only metric that was not better than the original was the perplexity, which shows that the pruned models are a lot less certain about their responses than the original model is. However, given that the other metrics were improved, this does not matter as much.

The results for testing if pre-training the model beforehand can be seen in Figures 5-7.

[2]Note: this happens when the model has max_new_tokens set to 0. Anything more will result in extra tokens that do not make sense towards the output. Since the pipeline could not have max_new_tokens to anything less than 1, the last token for each output was ignored.

6. Profiling Analysis

- Reading in models for the evaluation pipeline takes ∼40 seconds, which can lead to massive wait times if 77 models are being evaluated (leading to a wait time of about 51 minutes before the evaluation actually starts)

7. Reproducibility

The artifact can be found at https://github.com/lwashington3/cs595-2-project. By cloning the repository, Docker Compose can be used to the LTH module, along with containing scripts used to automatically generate the figures and tables in this paper. Running `docker compose run -it project` in the repo's root directory will run the ablation script with default values to generate masks for pruned models (pre-trained and not pre-trained), as well as the figures and tables during the evaluation. Other scripts are available in the scripts folder, along with documentation for how to set your own variables. With default values, the models should appear in the models folder, which will then be sorted by process name (default is "Ablation {UNIX timestamp that the script started}"), training status (trained or untrained), then the pruning percentage used. Within the `evaluation` folder, a folder with the same process name as before will contain a folder for each row in the test dataset. Each of those folders will contain the figures (saved as PNG, SVG, and LaTeX compatible-PGF formats), as well as the tables pre-processed in a LaTeX file. There will be two copies of each file's stem, one with `_baseline`, which has the baseline value added, such as Figures 1-4, and one only containing the pruned models, such as Figures 5-8.

8. Limitations

Currently, the pre-training process is limited by the size of the GPU. Without a GPU, the training could run for hours without visibly finishing the first example. Even then, a 32 GB Nvidia GeForce RTX 5090 pre-training Gemma could not fit any training rows onn the GPU, and the 80 GB GPU mentioned in Table 1 could only hold 6 rows.

9. Future Works

- Figure out how to have more control over the pre-training process so I can better control when training dataset rows are onloaded or offloaded from the GPU so more than 6 can work.
- Structurally remove transformer layers from the model and test how that affects it
- Figure out a better solution towards saving values (I could save the masked pre-trained value in a single file instead of having it distributed [this is only done because I don't want the same version of different files constantly written in different places]), but I not all values need to be written (save space)

10. References

[1] Eric Schwenker et al. *EXSCLAIM! – An automated pipeline for the construction of labeled materials imaging datasets from literature*. 2021. arXiv: 2103.10631 [cs.IR]. URL: https://arxiv.org/abs/2103.10631.

[2] Jonathan Frankle and Michael Carbin. "The Lottery Ticket Hypothesis: Training Pruned Neural Networks". In: *CoRR* abs/1803.03635 (2018). arXiv: 1803.03635. URL: https://arxiv.org/abs/1803.03635.

[3] Christopher Brix, Parnia Bahar, and Hermann Ney. "Successfully Applying the Stabilized Lottery Ticket Hypothesis to the Transformer Architecture". In: *CoRR* abs/2005.03454 (2020). arXiv: 2005.03454. URL: https://arxiv.org/abs/2005.03454.

[4] Lei Li et al. "Multimodal ArXiv: A Dataset for Improving Scientific Comprehension of Large Vision-Language Models". In: *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Ed. by Lun-Wei Ku, Andre Martins, and Vivek Srikumar. Bangkok, Thailand: Association for Computational Linguistics, Aug. 2024, pp. 14369–14387. DOI: 10.18653/v1/2024.acl-long.775. URL: https://aclanthology.org/2024.acl-long.775.

Appendix

A. Python Libraries

```
numpy==2.4.4
matplotlib==3.10.9
torch==2.11.0
datasets==4.8.5
transformers==5.8.0
evaluate[template]==0.4.6
safetensors==0.7.0
tqdm==4.67.3
huggingface_hub==1.14.0
accelerate==1.13.0
```

B. Results Tables

Tables 2 and 3 show the results for the same row of the dataset, grouped by pre-training status. The first row for both details the baseline model that was not pruned.

Table 2: Row 1 Results (Pre-trained before Pruning).

Trained	Pruning Percentage	Perplexity	ROGUE-L F1 Score (%)	TTFT (ms)	Run Time (ms)	Output Tokens	Time per Output Token (ms)
nan	0	**32.0966**	**100.0000%**	61.900	738.692	629	1.174
True	2.5%	390,428.4481	**100.0000%**	**9.181**	345.133	629	0.549
True	5.0%	442,413.3920	**100.0000%**	9.989	346.451	629	0.551
True	7.5%	568,070.0400	**100.0000%**	10.228	345.717	629	0.550
True	10.0%	776,459.6839	**100.0000%**	9.639	347.301	629	0.552
True	12.5%	685,223.2661	**100.0000%**	9.757	346.927	629	0.552
True	15.0%	1,061,294.5558	**100.0000%**	9.684	355.202	629	0.565
True	17.5%	1,450,617.6656	**100.0000%**	9.663	352.190	629	0.560
True	20.0%	1,362,729.1843	**100.0000%**	9.652	347.856	629	0.553
True	22.5%	2,391,664.2010	**100.0000%**	9.937	347.042	629	0.552
True	25.0%	2,391,664.2010	**100.0000%**	9.683	347.722	629	0.553
True	27.5%	2,710,110.5896	**100.0000%**	9.691	**344.353**	629	**0.547**
True	30.0%	3,704,281.9787	**100.0000%**	9.688	344.393	629	0.548
True	32.5%	5,389,698.4763	**100.0000%**	9.735	344.502	629	0.548
True	35.0%	5,737,304.1632	**100.0000%**	9.611	344.561	629	0.548
True	37.5%	4,756,392.2112	**100.0000%**	9.726	350.078	629	0.557
True	40.0%	5,737,304.1632	**100.0000%**	9.609	348.001	629	0.553
True	42.5%	8,347,728.3006	**100.0000%**	9.807	348.780	629	0.554
True	45.0%	6,920,509.8318	**100.0000%**	9.865	347.985	629	0.553
True	47.5%	7,366,844.3689	**100.0000%**	9.939	348.954	629	0.555
True	50.0%	11,409,991.7638	**100.0000%**	9.731	353.604	629	0.562
True	52.5%	8,347,728.3006	**100.0000%**	9.672	348.931	629	0.555
True	55.0%	18,811,896.1195	**100.0000%**	9.863	353.107	629	0.561
True	57.5%	12,929,214.5167	**100.0000%**	9.806	349.954	629	0.556
True	60.0%	24,154,952.7536	**100.0000%**	9.839	354.989	629	0.564
True	62.5%	16,601,440.0572	**100.0000%**	9.817	352.278	629	0.560
True	65.0%	12,929,214.5167	**100.0000%**	9.841	351.848	629	0.559
True	67.5%	8,886,110.5205	**100.0000%**	9.707	352.049	629	0.560
True	70.0%	24,154,952.7536	**100.0000%**	9.820	352.691	629	0.561
True	72.5%	35,145,248.8770	**100.0000%**	9.939	351.903	629	0.559
True	75.0%	157,510,077.7662	**100.0000%**	9.759	348.047	629	0.553
True	77.5%	259,690,215.5627	**100.0000%**	10.273	352.299	629	0.560
True	80.0%	27,371,147.3466	**100.0000%**	10.124	356.367	629	0.567
True	82.5%	27,371,147.3466	**100.0000%**	10.051	357.262	629	0.568
True	85.0%	622,964,442.1984	**100.0000%**	9.986	356.632	629	0.567
True	87.5%	65,659,969.1373	**100.0000%**	9.913	350.874	629	0.558
True	90.0%	428,156,782.1910	**100.0000%**	9.998	357.132	629	0.568
True	92.5%	27,371,147.3466	**100.0000%**	9.806	352.719	629	0.561
True	95.0%	2,110,636.2494	**100.0000%**	9.794	358.435	629	0.570
True	97.5%	1,544,174.4671	**100.0000%**	9.918	357.748	629	0.569

Table 3: Row 1 Results (No pre-training before pruning).

Trained	Pruning Percentage	Perplexity	ROGUE-L F1 Score (%)	TTFT (ms)	Run Time (ms)	Output Tokens	Time per Output Token (ms)
nan	0	**32.0966**	**100.0000%**	61.900	738.692	629	1.174
False	2.5%	304,065.9811	**100.0000%**	9.274	353.444	629	0.562
False	5.0%	344,551.8961	**100.0000%**	9.499	355.012	629	0.564
False	7.5%	344,551.8961	**100.0000%**	10.820	366.050	629	0.582
False	10.0%	304,065.9811	**100.0000%**	9.717	361.434	629	0.575
False	12.5%	344,551.8961	**100.0000%**	9.656	361.130	629	0.574
False	15.0%	323,676.5520	**100.0000%**	9.707	359.350	629	0.571
False	17.5%	323,676.5520	**100.0000%**	9.709	363.462	629	0.578
False	20.0%	323,676.5520	**100.0000%**	9.792	359.598	629	0.572
False	22.5%	366,773.5842	**100.0000%**	9.755	358.975	629	0.571
False	25.0%	323,676.5520	**100.0000%**	9.647	355.258	629	0.565
False	27.5%	323,676.5520	**100.0000%**	9.700	355.699	629	0.565
False	30.0%	304,065.9811	**100.0000%**	9.630	355.664	629	0.565
False	32.5%	344,551.8961	**100.0000%**	9.808	357.562	629	0.568
False	35.0%	323,676.5520	**100.0000%**	9.668	358.364	629	0.570
False	37.5%	366,773.5842	**100.0000%**	9.813	365.209	629	0.581
False	40.0%	323,676.5520	**100.0000%**	9.728	359.160	629	0.571
False	42.5%	344,551.8961	**100.0000%**	9.832	358.975	629	0.571
False	45.0%	344,551.8961	**100.0000%**	9.642	359.172	629	0.571
False	47.5%	344,551.8961	**100.0000%**	9.724	359.617	629	0.572
False	50.0%	323,676.5520	**100.0000%**	9.837	359.811	629	0.572
False	52.5%	323,676.5520	**100.0000%**	9.683	358.935	629	0.571
False	55.0%	304,065.9811	**100.0000%**	9.836	360.658	629	0.573
False	57.5%	323,676.5520	**100.0000%**	9.647	361.664	629	0.575
False	60.0%	323,676.5520	**100.0000%**	9.636	361.977	629	0.575
False	62.5%	344,551.8961	**100.0000%**	9.901	364.950	629	0.580
False	65.0%	344,551.8961	**100.0000%**	9.662	362.976	629	0.577
False	67.5%	304,065.9811	**100.0000%**	9.602	363.717	629	0.578
False	70.0%	323,676.5520	**100.0000%**	9.640	366.578	629	0.583
False	72.5%	304,065.9811	**100.0000%**	10.505	362.544	629	0.576
False	75.0%	323,676.5520	**100.0000%**	9.729	360.469	629	0.573
False	77.5%	323,676.5520	**100.0000%**	9.829	363.044	629	0.577
False	80.0%	344,551.8961	**100.0000%**	9.681	369.053	629	0.587
False	82.5%	323,676.5520	**100.0000%**	9.747	367.398	629	0.584
False	85.0%	304,065.9811	**100.0000%**	9.718	366.948	629	0.583
False	87.5%	344,551.8961	**100.0000%**	9.800	367.272	629	0.584
False	90.0%	323,676.5520	**100.0000%**	9.774	363.465	629	0.578
False	92.5%	304,065.9811	**100.0000%**	9.938	368.118	629	0.585
False	95.0%	344,551.8961	**100.0000%**	9.718	362.525	629	0.576
False	97.5%	323,676.5520	**100.0000%**	9.696	368.164	629	0.585

C. Results Figures

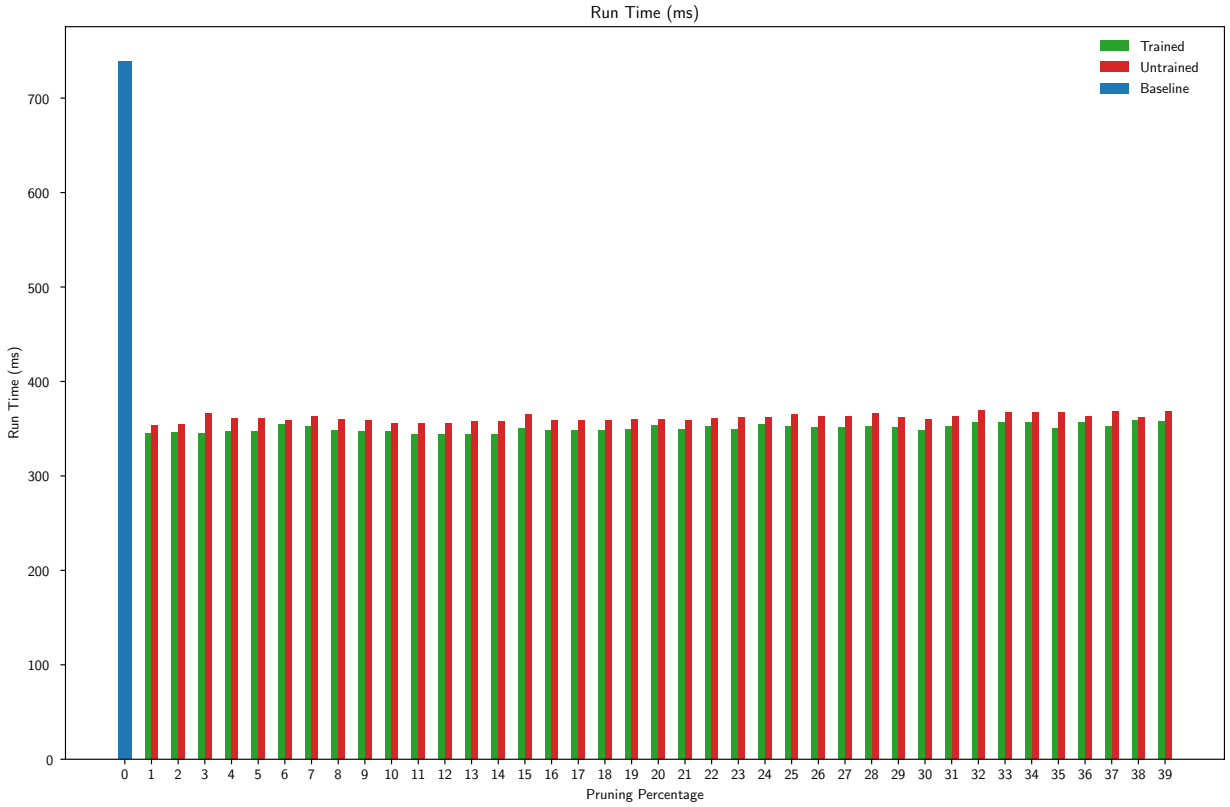

Figure 1: The total run time for each model.

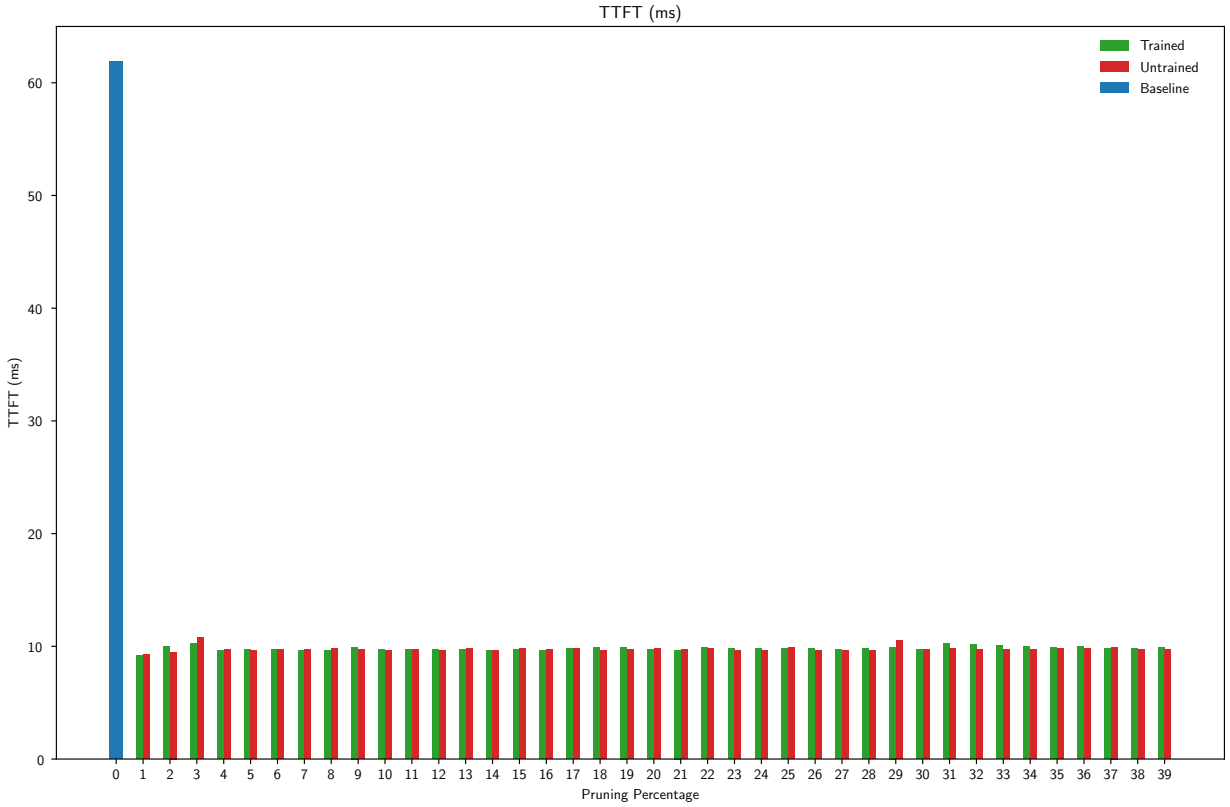

Figure 2: The amount of time it took the model to generate its first response token (Time to First Token).

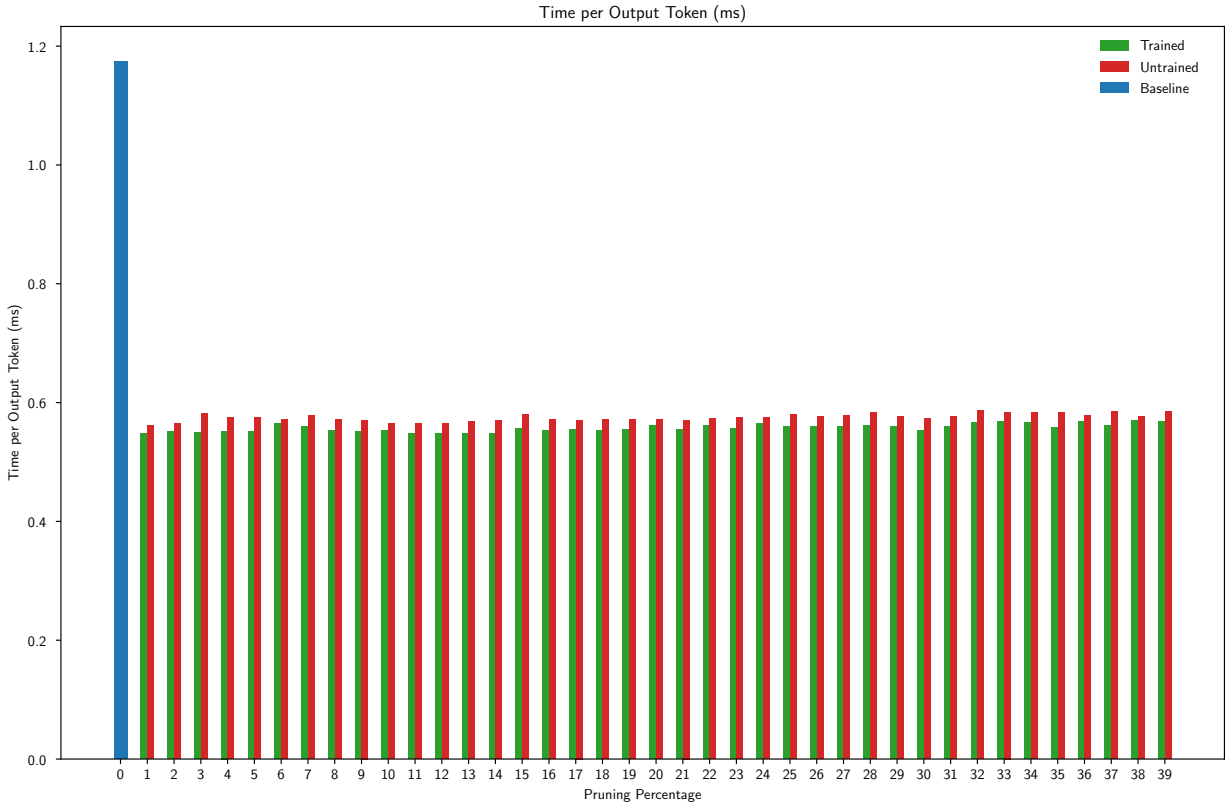

Figure 3: The average amount of time it took to generate each token in the response.

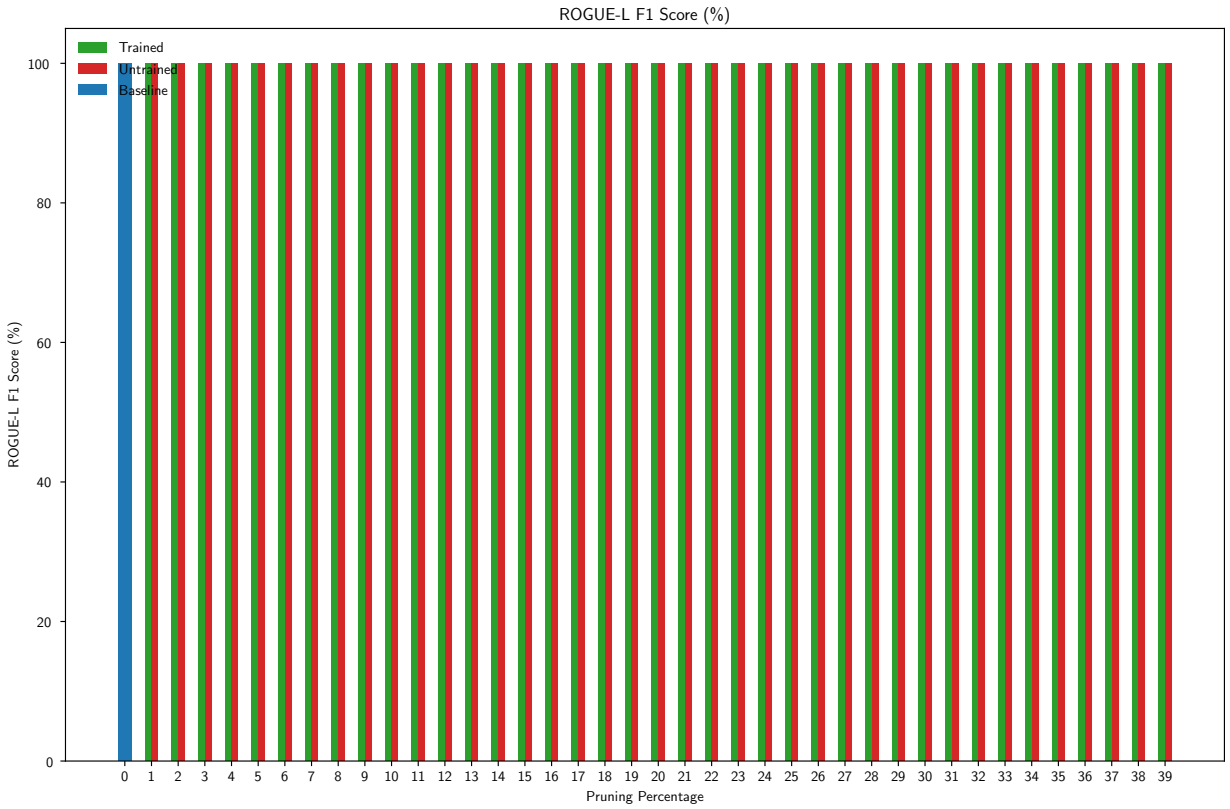

Figure 4: The ROGUE F1 Score for the Longest Common Subsequence.

D. Ablation Figures

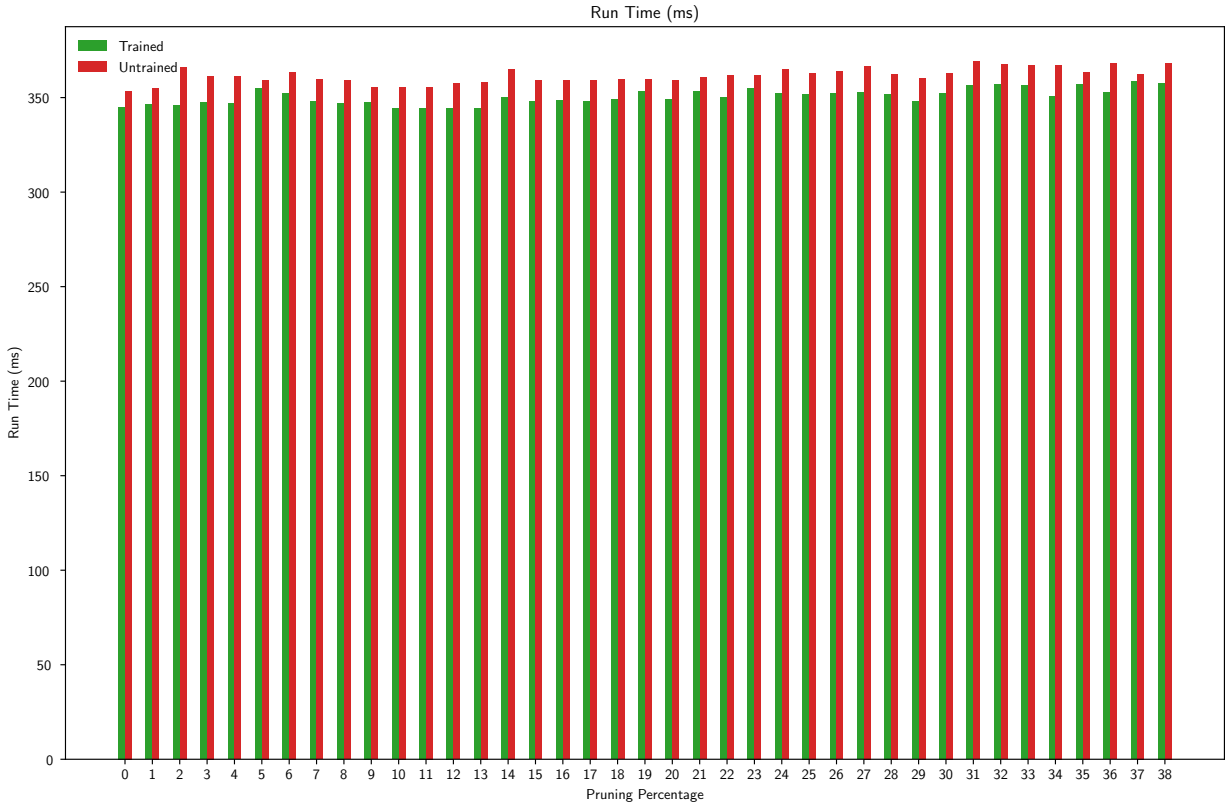

Figure 5: The total run time for each model (without the baseline).

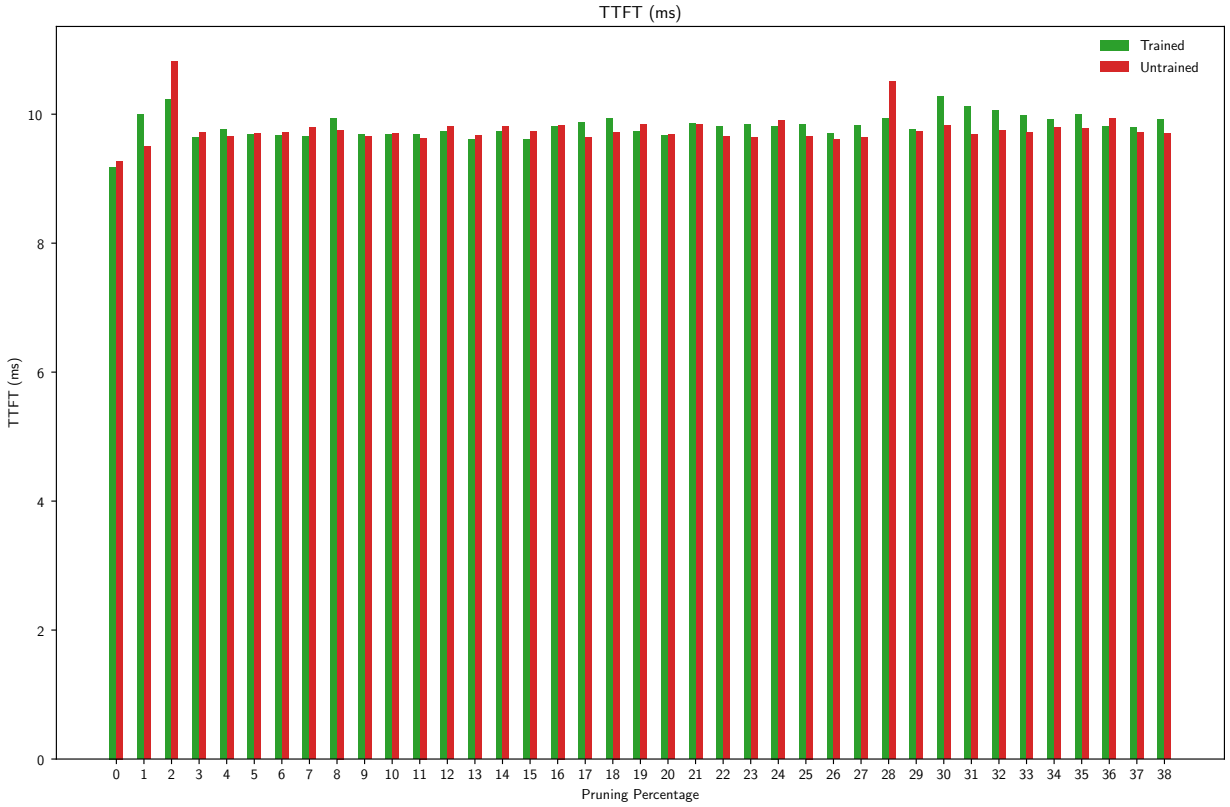

Figure 6: The amount of time it took the model to generate its first response token (Time to First Token) (without the baseline).

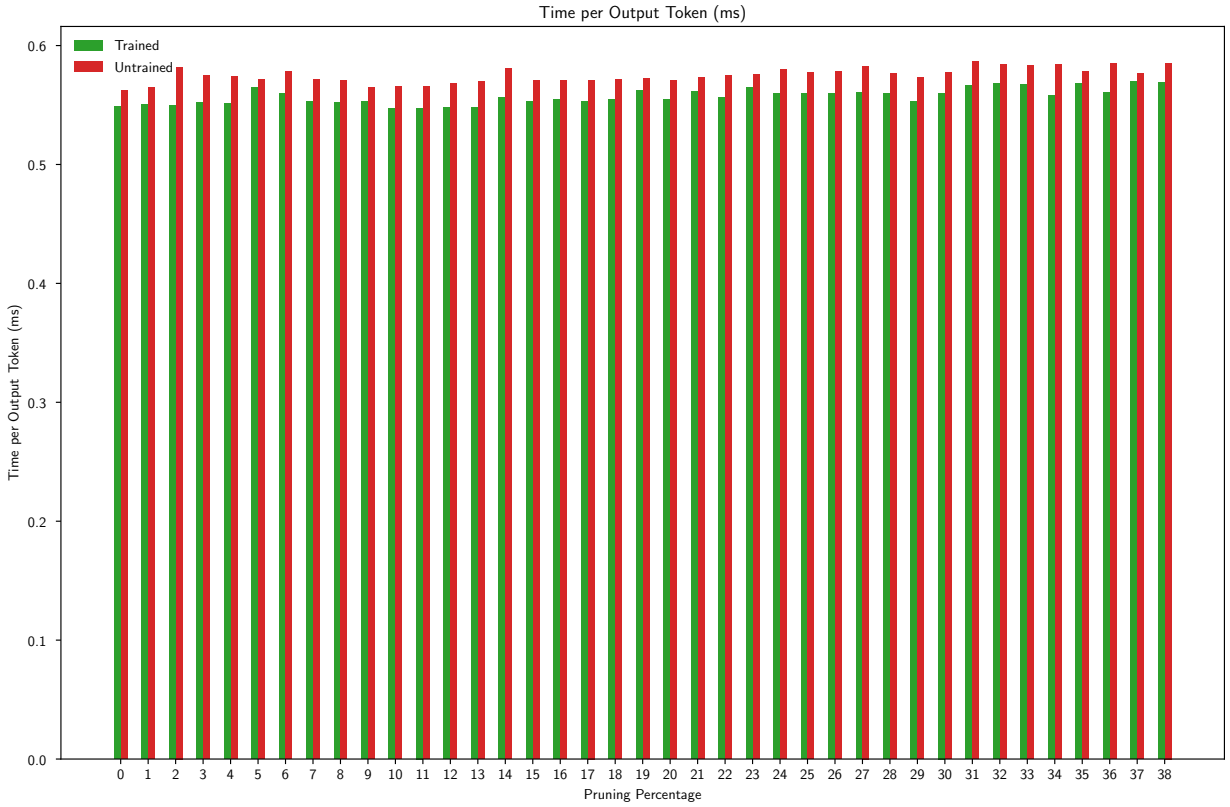

Figure 7: The average amount of time it took to generate each token in the response (without the baseline).

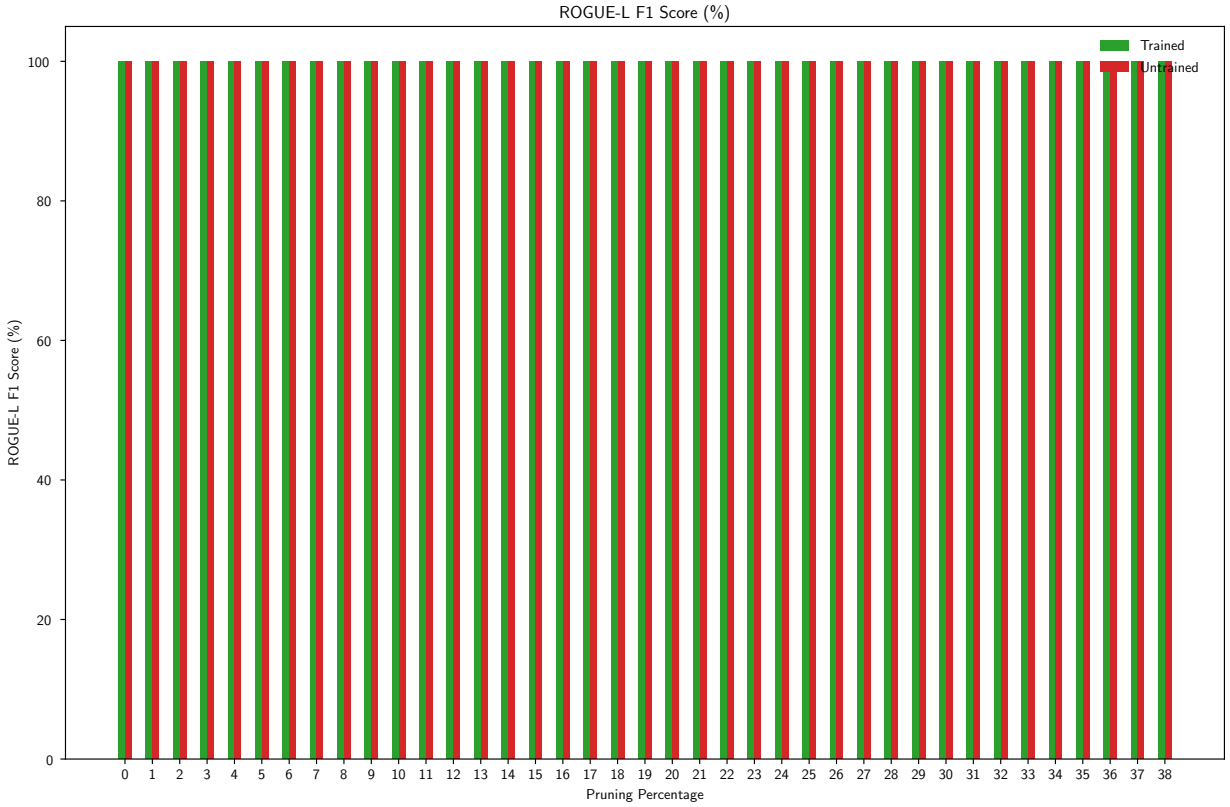

Figure 8: The ROGUE F1 Score for the Longest Common Subsequence (without the baseline).